



— SIERRA LEONE —

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

OBJECT ORIENTED PROGRAMMING – PROG211

Title : GROUP PRESENTATION
Issue Date : Week
Due Date : Week 11
Lecturer : Amandus Coker
Year/Semester : 2/3
Class : BIT 1202F

NO.	Name	Surname	ID	SIGN
1.	EMMANUEL WILLIAM	DIXON-CHARLIE	905004772	
2.	SAMUELLA MARY	KAMARA	905005109	
3.	Osman Aamir	Kamara	905005240	

Academic Honesty Policy Statement

We, hereby attest that contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Submitting Person's Signature:

Date: 5th/12/2025

(If submission is for a group work, all students' names MUST appear above and put down their signatures here)

LECTURER'S COMMENTS/GRADE:

for office use only upon receive
Remark

DATE :

TIME :

RECEIVER'S NAME :

1. Overview

Smart Library is a desktop application for Limkokwing University's central library to manage books, authors, members, book clubs and loan workflows.

The system enables librarians to perform catalog maintenance, manage members and loans, and operate book clubs; members can search, request and borrow books and join book clubs. The backend is implemented in Python (OOP) with PostgreSQL; GUI uses PyQt5; database connectivity via psycopg2.

2. Actors

- **Librarian:** full administrative privileges: manage books/authors/members, approving loans, run reports, manage book clubs, resolving overdue fines.
- **Member:** authenticated user who can search the catalog, request/borrow/return books, join/leave clubs, view personal loans.
- **System Administrator: (optional)** - installs, runs migrations, manages backups and user roles.
- **Guest: (unauthenticated)** - browse book catalog (read-only), view club descriptions.

3. Use Cases (high-level)

1. Login (role-based access)

- Actors: Librarian, Member
- Description: Authenticate using username/password; system enforces role-based UI/permissions.

2. Manage Books (CRUD)

- Actors: Librarian
- Description: Create, read, update and delete book records including title, ISBN, authors, categories, copies, publisher, year, and location.

3. Manage Authors (CRUD)

- Actors: Librarian

- Description: Create and maintain author profiles and link authors to books.

4. Managing Members (CRUD)

- Actors: Librarian
- Description: Register new members, update member info, set membership status.

5. Book Catalog Search / Filter / Sort

- Actors: Member, Librarian, Guest
- Description: Keyword search (title/author/ISBN), filters (genre, availability), sorting (most relevant, year, popularity).

6. Loan Workflow (Borrow/Return)

- Actors: Member, Librarian
- Description: Borrow a book (subject to business rules), return a book, record loan history, calculate due dates and overdue fines.

7. Book Club Management

- Actors: Librarian, Member
- Description: Create/manage clubs, enroll/remove members, post meetings/events, record club membership.

8. Dashboard / Reports

- Actors: Librarian
- Description: View most borrowed books, active clubs, overdue loans list, member activity.

9. Business Rules Enforcement

- Max 3 concurrent loans per member.
- Standard loan duration 7 days (configurable).
- Only available copies can be borrowed.
- Role-based access control: Librarian vs Member.

4. Functional Requirements (CRUD & workflows)

Books

- FR-B1: System shall allow librarian to create book records with fields: id, title, isbn, publisher, year, total copies, available copies, location, summary, categories.
- FR-B2: System shall link books to one or more authors.
- FR-B3: System shall permit searching books by title, author, ISBN or keyword.

Authors

- FR-A1: System shall provide CRUD operations for authors including name, biography, DOB.

Members

- FR-M1: System shall register members with fields: id, name, email, phone, username, password hash, role, membership status, join date.
- FR-M2: System shall authenticate username/password and produce a session with role-based permissions.

Loans

- FR-L1: System shall record loans with fields: id, book_id, member_id, loan_date, due_date, return_date, status (borrowed, returned, overdue).
- FR-L2: System shall prevent loans if member has ≥ 3 active loans.
- FR-L3: System shall set $\text{due_date} = \text{loan_date} + 7 \text{ days}$ (configurable).
- FR-L4: System shall decrement available_copies when a loan is issued and increment on return.
- FR-L5: System shall allow librarians to mark returns and compute overdue days.

Book Clubs

- FR-C1: Create/update/delete club records (name, description, leader_id).
- FR-C2: Allow members to join/leave clubs.

Dashboard/Reports

- FR-R1: Present most borrowed books (top 10), active clubs, list of overdue loans.

5. Non-Functional Requirements (NFRs)

- **Performance:** Common queries (search, loan) respond within 1 second for up to 10k records on target hardware.
- **Security:** Passwords hashed (bcrypt). Use parameterized queries (psycopg2) to prevent SQL injection. Role-based UI restrictions.
- **Usability:** Desktop UI should be intuitive; login, search and loan flows require max 3 clicks.
- **Availability:** Local installation expected; database backup instructions provided.
- **Maintainability:** Use modular OOP, clear DAOs, unit tests, and Git with branch workflow.

6. Wireframes / Sketches (textual)

- **Login Screen:** fields: Username, Password, Login button, "Forgot password" (optional). Role-based redirect to Dashboard (Librarian vs Member).
- **Book Catalog Screen:** Search bar, filter dropdowns (genre, availability), results list with pagination; each item has "Borrow" (if available) or "Request" button.
- **Loan Screen:** Member loan history, active loans with due dates and "Return" action.
- **Book Club Screen:** List of clubs, Join/Leave button, Create/Edit club (Librarian).
- **Dashboard:** KPI cards (Active loans, Overdue, Most borrowed), tables (top 10 books, overdue loans).

7. Acceptance Criteria

- Requirements document included and uploaded.
- Use-case descriptions, actor mapping, functional and non-functional requirements, and basic wireframes.

Database Design — ERD description + PostgreSQL DDL (copy/paste SQL)

ERD (entities & relationships summary)

- Book (1) — (M) BookAuthor — (M) Author (many-to-many)
- Book (1) — (M) Loan (one book copy per loan)
- Member (1) — (M) Loan
- Member (M) — (M) BookClubMembership — (1) BookClub
- User is used for authentication; members and librarians are Users with a Role.

PostgreSQL DDL

-- PostgreSQL DDL for SmartLibrary

```
CREATE TABLE roles (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL UNIQUE  
);
```

```
INSERT INTO roles (name) VALUES ('Librarian'), ('Member');
```

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(100) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name VARCHAR(200) NOT NULL,  
    email VARCHAR(200),
```

```
phone VARCHAR(50),  
role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE RESTRICT,  
status VARCHAR(20) DEFAULT 'active',  
created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE authors (  
    id SERIAL PRIMARY KEY,  
    full_name VARCHAR(200) NOT NULL,  
    biography TEXT,  
    birth_date DATE  
);
```

```
CREATE TABLE books (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(500) NOT NULL,  
    isbn VARCHAR(30),  
    publisher VARCHAR(200),  
    year INT,  
    total_copies INT DEFAULT 1 CHECK (total_copies >= 0),  
    available_copies INT DEFAULT 1 CHECK (available_copies >= 0),  
    location VARCHAR(200),  
    summary TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE book_authors (  
    book_id INT REFERENCES books(id) ON DELETE CASCADE,  
    author_id INT REFERENCES authors(id) ON DELETE CASCADE,  
    PRIMARY KEY (book_id, author_id)  
);
```

```
CREATE TABLE book_clubs (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(200) NOT NULL,  
    description TEXT,  
    leader_id INT REFERENCES users(id) -- librarian or member  
);
```

```
CREATE TABLE book_club_membership (  
    club_id INT REFERENCES book_clubs(id) ON DELETE CASCADE,  
    user_id INT REFERENCES users(id) ON DELETE CASCADE,  
    joined_at TIMESTAMP DEFAULT NOW(),  
    PRIMARY KEY (club_id, user_id)  
);
```

```
CREATE TABLE loans (  
    id SERIAL PRIMARY KEY,  
    book_id INT REFERENCES books(id) ON DELETE RESTRICT,  
    user_id INT REFERENCES users(id) ON DELETE RESTRICT,  
    loan_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    due_date DATE NOT NULL,  
    return_date DATE,  
    status VARCHAR(20) NOT NULL DEFAULT 'borrowed', -- borrowed, returned,  
    overdue  
    CONSTRAINT chk_due_after_loan CHECK (due_date >= loan_date)  
);
```

-- Indexes for performance

```
CREATE INDEX idx_books_title ON books USING btree (lower(title));  
CREATE INDEX idx_books_isbn ON books (isbn);  
CREATE INDEX idx_users_username ON users (username);  
CREATE INDEX idx_loans_user_status ON loans (user_id, status);
```


Sample data inserts (small set for testing)

-- sample user accounts (password_hash placeholders)

```
INSERT INTO users (username, password_hash, full_name, email, role_id)
VALUES
('librarian1', '$2b$12$placeholderhash', 'Librarian One', 'lib1@example.com', 1),
('member1', '$2b$12$placeholderhash', 'Member One', 'm1@example.com', 2);
```

```
INSERT INTO authors (full_name) VALUES ('Jane Austen'), ('George Orwell');
```

```
INSERT INTO books (title, isbn, publisher, year, total_copies, available_copies,
location)
```

```
VALUES
('Pride and Prejudice', '9780141199078', 'Penguin', 1813, 3, 3, 'Shelf A3'),
('1984', '9780451524935', 'Secker & Warburg', 1949, 2, 2, 'Shelf B1');
```

```
INSERT INTO book_authors (book_id, author_id) VALUES (1,1), (2,2);
```

ERD & UML class diagrams (PlantUML text — paste into PlantUML server or local tool)

PlantUML ERD (simplified)

```
@startuml
```

```
entity USERS {
```

```
    *id : int
```

```
    *username : varchar
```

```
    *password_hash : varchar
```

```
    *full_name : varchar
```

```
    *role_id : int
```

```
}
```

```
entity ROLES {
```

```
*id : int
*name : varchar
}
entity BOOKS {
  *id : int
  *title : varchar
  *isbn : varchar
  *total_copies : int
  *available_copies : int
}
entity AUTHORS {
  *id : int
  *full_name : varchar
}
entity BOOK_AUTHORS {
  *book_id : int
  *author_id : int
}
entity LOANS {
  *id : int
  *book_id : int
  *user_id : int
  *loan_date : date
  *due_date : date
  *return_date : date
}
entity BOOK_CLUBS {
  *id : int
  *name : varchar
  *leader_id : int
}
entity BOOK_CLUB_MEMBERSHIP {
```

```

    *club_id : int
    *user_id : int
}
USERS }|..|| ROLES
BOOKS ||--o{ BOOK_AUTHORS
AUTHORS ||--o{ BOOK_AUTHORS
BOOKS ||--o{ LOANS
USERS ||--o{ LOANS
BOOK_CLUBS ||--o{ BOOK_CLUB_MEMBERSHIP
USERS ||--o{ BOOK_CLUB_MEMBERSHIP
@enduml

```

PlantUML class diagram for Python classes

```

@startuml
class User {
    -id:int
    -username:str
    -password_hash:str
    -role:str
    +authenticate(password)->bool
}
class Member extends User
class Librarian extends User
class Author {
    -id:int
    -full_name:str
    -biography:str
}
class Book {
    -id:int

```

```
-title:str
-isbn:str
-total_copies:int
-available_copies:int
+is_available()->bool
}
```

```
class Loan {
  -id:int
  -book:Book
  -user:User
  -loan_date:date
  -due_date:date
  -return_date:date
  +is_overdue()->bool
}
```

```
class BookClub {
  -id:int
  -name:str
  -members:list
  +add_member(user)
  +remove_member(user)
}
```

User <|-- Member

User <|-- Librarian

Book "1" *-- "0..*" Author : authors

Book "1" *-- "0..*" Loan : loans

User "1" *-- "0..*" Loan : loans

BookClub "1" *-- "0..*" User : members

@enduml

Backend - Python (OOP) with psycopg2, DAOs, and business rules requirements (for Python environment)

psycopg2-binary==2.9.6

PyQt5==5.15.7

bcrypt==4.0.1

python-dotenv==1.0.0

db.py — connection helper

```
# db.py
```

```
import os
```

```
import psycopg2
```

```
from psycopg2.extras import RealDictCursor
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
DB_CONFIG = {
```

```
    'host': os.getenv('DB_HOST', 'localhost'),
```

```
    'port': int(os.getenv('DB_PORT', 5432)),
```

```
    'dbname': os.getenv('DB_NAME', 'smartlibrary'),
```

```
    'user': os.getenv('DB_USER', 'postgres'),
```

```
    'password': os.getenv('DB_PASS', 'password')
```

```
}
```

```
def get_conn():
```

```
    conn = psycopg2.connect(
```

```
        host=DB_CONFIG['host'],
```

```
        port=DB_CONFIG['port'],
```

```
        dbname=DB_CONFIG['dbname'],
```

```
        user=DB_CONFIG['user'],
```

```
        password=DB_CONFIG['password']
```

```
)  
return conn
```

models.py — simple domain classes

```
# models.py  
from dataclasses import dataclass  
from datetime import date, timedelta
```

```
@dataclass  
class User:  
    id:int  
    username:str  
    full_name:str  
    role:str
```

```
@dataclass  
class Author:  
    id:int  
    full_name:str  
    biography:str=None
```

```
@dataclass  
class Book:  
    id:int  
    title:str  
    isbn:str  
    total_copies:int  
    available_copies:int  
  
    def is_available(self):  
        return self.available_copies > 0
```

```
@staticmethod
def get_by_username(username):
    conn = get_conn()
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT u.id, u.username, u.full_name, r.name as role "
                        "FROM users u JOIN roles r ON u.role_id=r.id WHERE "
                        "u.username=%s", (username,))
```

```

        row = cur.fetchone()
        if row:
            return User(id=row[0], username=row[1], full_name=row[2],
role=row[3])
        finally:
            conn.close()
    return None

```

class BookDao:

```

    @staticmethod
    def get_by_id(book_id):
        conn = get_conn()
        try:
            with conn.cursor(cursor_factory=None) as cur:
                cur.execute("SELECT id, title, isbn, total_copies, available_copies
FROM books WHERE id=%s", (book_id,))
                r = cur.fetchone()
                if r:
                    return Book(id=r[0], title=r[1], isbn=r[2], total_copies=r[3],
available_copies=r[4])
        finally:
            conn.close()
    return None

```

@staticmethod

```

def search(term, limit=50):
    conn = get_conn()
    try:
        with conn.cursor(cursor_factory=None) as cur:
            like = f"%{term.lower()}%"
            cur.execute("SELECT id, title, isbn, total_copies, available_copies
FROM books WHERE lower(title) LIKE %s OR isbn LIKE %s LIMIT %s",

```



```

        (like, like, limit))
    rows = cur.fetchall()
    return [Book(id=r[0], title=r[1], isbn=r[2], total_copies=r[3],
available_copies=r[4]) for r in rows]
    finally:
        conn.close()

```

```

class LoanDao:

```

```

    @staticmethod
    def active_loans_count_for_user(user_id):
        conn = get_conn()
        try:
            with conn.cursor() as cur:
                cur.execute("SELECT COUNT(*) FROM loans WHERE user_id=%s
AND status='borrowed'", (user_id,))
            return cur.fetchone()[0]
        finally:
            conn.close()

```

```

    @staticmethod
    def create_loan(book_id, user_id):
        # business rules enforced here
        from db import get_conn
        loan_date = date.today()
        due_date = loan_date + timedelta(days=LOAN_PERIOD_DAYS)
        conn = get_conn()
        try:
            with conn:
                with conn.cursor() as cur:
                    # Check availability
                    cur.execute("SELECT available_copies FROM books WHERE id=%s
FOR UPDATE", (book_id,))

```

```

        row = cur.fetchone()
        if not row:
            raise Exception("Book not found")
        available = row[0]
        if available <= 0:
            raise Exception("No available copies")

        # Check member active loans
        cur.execute("SELECT COUNT(*) FROM loans WHERE user_id=%s
AND status='borrowed'", (user_id,))
        active = cur.fetchone()[0]
        if active >= MAX_ACTIVE_LOANS:
            raise Exception("Member has reached maximum active loans")

        # Create loan
        cur.execute("INSERT INTO loans (book_id, user_id, loan_date,
due_date, status) VALUES (%s,%s,%s,%s,%s) RETURNING id",
                    (book_id, user_id, loan_date, due_date, 'borrowed'))
        loan_id = cur.fetchone()[0]

        # Decrement available
        cur.execute("UPDATE books SET available_copies =
available_copies - 1 WHERE id=%s", (book_id,))
        return loan_id

    finally:
        conn.close()

    @staticmethod
    def return_loan(loan_id):
        conn = get_conn()
        try:
            with conn:

```

```

        with conn.cursor() as cur:
            cur.execute("SELECT book_id, status FROM loans WHERE id=%s
FOR UPDATE", (loan_id,))
            r = cur.fetchone()
            if not r:
                raise Exception("Loan not found")
            book_id, status = r
            if status != 'borrowed':
                raise Exception("Loan already returned or closed")
            cur.execute("UPDATE loans SET return_date = CURRENT_DATE,
status='returned' WHERE id=%s", (loan_id,))
            cur.execute("UPDATE books SET available_copies =
available_copies + 1 WHERE id=%s", (book_id,))
            return True
    finally:
        conn.close()

```

Example usage (script)

```

# example_usage.py
from dao import UserDao, BookDao, LoanDao

user = UserDao.get_by_username('member1')
print('user', user)

books = BookDao.search('1984')
print('search results', books)

# borrow attempt
try:
    loan_id = LoanDao.create_loan(book_id=2, user_id=user.id)
    print('Loan created id', loan_id)

```

```
except Exception as e:  
    print('Loan failed:', e)
```

PyQt5 GUI prototype (runnable skeleton)

```
# app.py  
  
import sys  
  
from PyQt5.QtWidgets import (QApplication, QWidget, QMainWindow,  
                              QVBoxLayout, QLineEdit,  
                              QPushButton, QLabel, QListWidget, QMessageBox,  
                              QHBoxLayout)  
  
from dao import UserDao, BookDao, LoanDao  
  
class LoginWindow(QWidget):  
    def __init__(self, on_success):  
        super().__init__()  
        self.on_success = on_success  
        self.init_ui()  
  
    def init_ui(self):  
        self.setWindowTitle('SmartLibrary Login')  
        layout = QVBoxLayout()  
        self.username = QLineEdit()  
        self.username.setPlaceholderText('Username')  
        self.password = QLineEdit()  
        self.password.setEchoMode(QLineEdit.Password)  
        self.password.setPlaceholderText('Password')  
        self.login_btn = QPushButton('Login')  
        self.login_btn.clicked.connect(self.attempt_login)  
        layout.addWidget(QLabel('SmartLibrary Login'))  
        layout.addWidget(self.username)  
        layout.addWidget(self.password)  
        layout.addWidget(self.login_btn)
```

```
self.setLayout(layout)
```

```
def attempt_login(self):
```

```
    u = UserDao.get_by_username(self.username.text())
```

```
    if u:
```

```
        # skipping real password check in this stub
```

```
        self.on_success(u)
```

```
        self.close()
```

```
    else:
```

```
        QMessageBox.warning(self, 'Login failed', 'Invalid username')
```

```
class CatalogWindow(QMainWindow):
```

```
    def __init__(self, user):
```

```
        super().__init__()
```

```
        self.user = user
```

```
        self.setWindowTitle('SmartLibrary Catalog')
```

```
        self.init_ui()
```

```
    def init_ui(self):
```

```
        central = QWidget()
```

```
        v = QVBoxLayout()
```

```
        self.search_input = QLineEdit()
```

```
        self.search_input.setPlaceholderText('Search title or ISBN')
```

```
        self.search_btn = QPushButton('Search')
```

```
        self.search_btn.clicked.connect(self.do_search)
```

```
        self.list_widget = QListWidget()
```

```
        self.borrow_btn = QPushButton('Borrow Selected')
```

```
        self.borrow_btn.clicked.connect(self.borrow_selected)
```

```
        v.addWidget(QLabel(f'Logged in as: {self.user.full_name}
```

```
{self.user.role}'))
```

```
        v.addWidget(self.search_input)
```

```
        v.addWidget(self.search_btn)
```

```
v.addWidget(self.list_widget)
v.addWidget(self.borrow_btn)
central.setLayout(v)
self.setCentralWidget(central)
```

```
def do_search(self):
    term = self.search_input.text()
    results = BookDao.search(term)
    self.list_widget.clear()
    for b in results:
        self.list_widget.addItem(f'{b.id} | {b.title} | avail: {b.available_copies}')
```

```
def borrow_selected(self):
    item = self.list_widget.currentItem()
    if not item:
        QMessageBox.warning(self, 'No selection', 'Select a book first')
        return
    book_id = int(item.text().split('|')[0].strip())
    try:
        loan_id = LoanDao.create_loan(book_id=book_id, user_id=self.user.id)
        QMessageBox.information(self, 'Success', f'Loan created: {loan_id}')
        self.do_search()
    except Exception as e:
        QMessageBox.critical(self, 'Error', str(e))
```

```
def main():
    app = QApplication(sys.argv)
    def on_success(user):
        win = CatalogWindow(user)
        win.show()
    login = LoginWindow(on_success)
    login.show()
```

```
sys.exit(app.exec_())
```

```
if __name__ == '__main__':  
    main()
```

README (setup, run, migrations)

SmartLibrary — Desktop Application

Requirements

- Python 3.10+
- PostgreSQL 12+
- pip

Dependencies

pip install -r requirements.txt

requirements.txt:

psycopg2-binary==2.9.6

PyQt5==5.15.7

bcrypt==4.0.1

python-dotenv==1.0.0

Setup

1. Create PostgreSQL database:

- psql -U postgres
- CREATE DATABASE smartlibrary;
- CREATE USER sl_user WITH PASSWORD 'yourpassword';
- GRANT ALL PRIVILEGES ON DATABASE smartlibrary TO sl_user;

2. Run DDL (copy SQL from schema.sql and execute in psql).

3. Create .env file:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=smartlibrary
DB_USER=sl_user
DB_PASS=yourpassword
```

4. Insert sample data (optional) using sample SQL provided.

```
## Run application (development)
python smartlibrary/gui/app.py
```

Notes

- Passwords in sample users are placeholder; use bcrypt to hash real passwords.
- For production, configure database backups and secure the .env file.

Poster / Infographic (HTML + text)

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>SmartLibrary - Poster</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 900px; margin: 20px auto; }
    .header { text-align:center; }
    .card { border:1px solid #ddd; padding:12px; margin:8px 0; border-
radius:8px; }
    h1{margin:6px}
    .cols { display:flex; gap:12px; }
    .col { flex:1; }
    ul { margin:6px 0; }
    .kpi { display:flex; gap:8px; }
    .k { background:#f6f6f6; padding:10px; border-radius:6px; flex:1; text-
align:center; }
  </style>
</head>
<body>
  <div class="header">
```



```

    <h1>SmartLibrary</h1>
    <p>Library Management System — Limkokwing University</p>
</div>

<div class="card">
    <h2>What it does</h2>
    <p>Search catalog, borrow/return books, manage members and book clubs,
    dashboard and reports.</p>
</div>

<div class="cols">
    <div class="col card">
        <h3>Actors</h3>
        <ul><li>Librarian (admin)</li><li>Member</li><li>Guest</li></ul>
        <h3>Key Features</h3>
        <ul><li>Role-based login</li><li>Catalog search & filters</li><li>Loan
workflows (7-day loans)</li></ul>
    </div>
    <div class="col card">
        <h3>Stack</h3>
        <ul><li>PyQt5 (Desktop GUI)</li><li>Python OOP (Business
logic)</li><li>PostgreSQL + psycopg2</li></ul>
        <h3>Business Rules</h3>
        <ul><li>Max 3 active loans per member</li><li>Loan period = 7
days</li></ul>
    </div>
</div>

<div class="card">
    <h3>Contact / Team</h3>
    <p>[Osman Aamir Kamara, osmank617@gmail.com]</p>
</div>
</body>
</html>

```